

Low-Cost Indoor Navigation Benchmark: Research Protocol

Project: ubot, Unipod-Robotics · **Target venue:** IEEE/RSJ IROS 2027 (submission deadline 1 March 2027; 6 pages + 2 references) · **Protocol version:** 1.3, 1 October 2026 · **Code:**

`src/ubot/ubot_bench` and `src/ubot/ubot_worlds` on branch `feat/nav-benchmark`.

This document is the single reference for how the study is run. Anything that changes the design after the pilot (section 9.3) is recorded in the change log at the end, with the date and the reason. The analysis plan (section 10) is pre-registered on OSF before the real-robot campaign starts. After that it changes only through a dated, justified amendment.

1. Question and hypotheses

Core question. How much localisation and navigation performance does a differential-drive robot lose, and where, when it uses sub-\$100 range sensors instead of a reference-grade LiDAR? In both cases the robot also has wheel odometry and an IMU.

R Q	Question	Primary measure	Hypothesis (direction fixed before data)
R Q 1	Accuracy gap: how far are budget stacks from the reference in localisation and mapping?	SLAM ATE RMSE, map F1	H1: every budget LiDAR has a higher median ATE than REF in every world. The gap is largest in the warehouse, where open space exceeds the 12 m range.
R Q 2	Navigation gap: does the accuracy gap change task outcomes?	SPL, success rate	H2: in the arena and house, success of the budget LiDARs is within 10 percentage points of REF. The gap grows with world size.
R Q 3	Where does it break?	Change from nominal per condition; failure taxonomy	H3: glass (C1) causes collisions for every LiDAR stack. Sensor degradation (C3) affects the 450-beam sensors more than REF. Low light (C4) does not affect the stereo-depth stack in simulation (a stated limitation, section 12).
R Q 4	Cost-efficiency: which stack is Pareto-optimal on price, compute and accuracy?	Price vs ATE / SPL; CPU %	H4: at least one budget LiDAR is on the Pareto front.
R Q 5	Validity: does the calibrated simulation predict the real-robot results?	Sim-vs-Real Correlation Coefficient (SRCC) [Kadian 2020], Kendall τ of stack rankings	H5: SRCC ≥ 0.7 for success and ATE across the budget stacks in the arena. If it is lower, the sim-only REF comparison (RQ1) is reported as indicative only.

Why RQ5 matters. There is no physical reference-grade LiDAR in this study (section 4.2). The reference exists only in simulation. Our simulated budget sensors are measured against their real counterparts in the same arena and missions, and that agreement is what licenses reading the sim reference as an upper bound.

2. Claimed contributions

1. A reproducible cost-versus-accuracy benchmark for 2D indoor navigation on commodity hardware. It covers four simulated worlds, a physical arena with a sim replica, and datasheet-derived sensor models, and it releases the code, worlds, missions, ground-truth maps and data.

2. A quantified accuracy and task gap between three sub-\$150 sensors and a reference LiDAR, with paired statistics over identical missions and noise seeds.
3. A condition-level failure analysis (glass, dynamic obstacles, degradation, low light) and a failure-mode taxonomy.
4. A measured sim-to-real predictivity (SRCC) for the benchmark itself.

The novelty is framed against the related work below. This is a benchmark and dataset paper, not a new algorithm.

3. Related work (verified references only)

- **SLAM evaluation metrics.**
 - ATE and RPE [Sturm 2012], computed with evo [Grupp 2017].
 - Relative-relations evaluation [Kümmerle 2009].
- **Navigation benchmarks.**
 - BARN: 300 simulated obstacle courses ordered by difficulty metrics [Perille 2020].
 - SPL for embodied navigation [Anderson 2018].
 - TaskSLAM-Bench compares visual and LiDAR SLAM by task-driven precision (repeatability) [Du 2025]. It is the closest related work. We differ by (a) holding the SLAM method fixed and varying the *sensor stack at a price point*, (b) adding the cost axis, (c) adding degradation conditions, and (d) measuring sim-to-real predictivity.
- **Low-cost LiDAR characterisation.** Calibrated comparisons of 2D LiDARs, including the RPLIDAR A1 [Ziębiński 2025], characterise the sensor, not the navigation task.
- **Glass.** LiDARs largely see through or specularly reflect off glass.
 - Cartographer_glass detects glass from intensity spikes [Weerakoon 2022].
 - TOPGN uses point-cloud intensity to detect transparent obstacles [Weerakoon 2024].
 - Our C1 condition quantifies the consequence for unmodified low-cost stacks.
- **Software under test.** Nav2 [Macenski 2020] and SLAM Toolbox [Macenski 2021].
- **Sim-to-real.** SRCC [Kadian 2020].

Before submission the literature search is repeated, particularly for 2025–2027 low-cost SLAM and navigation benchmarks. Any new close work goes into this section and the change log.

4. System under test

4.1 Robot (fixed across all conditions)

- **Chassis.** ubot "Inspection Robot 01": 4-wheel skid-steer, 0.357 × 0.232 m body, wheel radius 0.0325 m.
- **Drive.**
 - Front wheels are commanded and carry encoders; the rear wheels mirror them.
 - `ros2_control diff_drive_controller`: `wheel_separation` 0.264204 in sim; 0.4192 on the real robot (effective skid-steer value).
- **IMU.** BNO085; the EKF fuses its yaw rate.
 - Real gyro noise (E2 still test) is $\sigma = 0.0019$ rad/s, and the sim uses the same value.
- **EKF.** `robot_localization` fuses wheel vx and yaw rate plus IMU yaw rate (`sim_ekf.yaml` = `real_ekf.yaml`).

- **In-place turns pivot on an axle (measured).** When the simulated robot turns on the spot, it rotates about the midpoint of its **front axle**, not its centre. The ground-truth path is a circle of radius 0.127 m centred at (+0.127, +0.005) m in the body frame, so a 180° turn moves `base_footprint` by 0.25 m. This is correct Coulomb-friction physics for a skid-steer. The sideways scrub forces of the two axles must balance, so the more heavily loaded axle (centre of mass at $x = +0.013$ m) stops sliding and becomes the pivot. Moving the centre of mass 16 mm rearward moves the pivot to the rear axle. The pivot does not change with collision shape (cylinder or sphere), friction direction, wheel declaration order, mimic-joint or explicitly driven rear wheels, or a finite wheel torque.
 - The diff-drive odometry model assumes a centre pivot, so this motion is invisible to the EKF and has to be corrected by the range sensor. It is identical for every stack.
 - It is a real property of skid-steer robots, so the real robot's pivot point is measured with the ground-truth rig (section 7.3) and reported with the RQ5 comparison.
- **Compute.**
 - Raspberry Pi 5 runs drivers, control and the EKF.
 - A laptop runs SLAM, Nav2 and the ground-truth pipeline over the same LAN; this is recorded.
- **Software.** ROS 2 Jazzy, Gazebo Harmonic (gz-sim 8, DART), Nav2 (Jazzy), `slam_toolbox` (Jazzy), `robot_localization`. Every trial records the exact git commit (section 11).

4.2 Sensor stacks (the independent variable)

Values are from the datasheets and are the ones used by the sim noise models (`ubot_description/config/sensor_profiles.yaml`). Prices are approximate (September 2026) and must be replaced with dated vendor quotes before submission.

Stack	Sensor	FOV	Rate	Beams	Range	Noise model (σ)	Price	Real?
REF	Hokuyo UST-10LX	270°	40 Hz	1081	0.06–10 m	20 mm (± 40 mm read as 2σ)	~\$1,600	sim only
MS200	Oradar MS200	360°	10 Hz	450	0.03–12 m	4 mm < 2 m, 15 mm $\geq$ 2 m (1σ , manual) + per-run bias $\pm 10/\pm 20$ mm	~\$60	yes
LD06	LDROBOT LD06	360°	10 Hz	450	0.02–12 m	22.5 mm (± 45 mm as 2σ)	~\$80	yes
OAK-D	Luxonis OAK-D Lite (stereo depth $\rightarrow$ scan)	69°	15–30 Hz	139 bins	0.2–8 m	$\sigma = 0.0025 \cdot r^2$ (fitted to "<2 % at 4 m")	~\$149	yes

- **Odometry-only baseline (B0).** EKF only, no range sensor. It is not a separate run: every mapping trial logs the EKF trajectory, so B0 is measured in all of them on identical motion.
- **Depth $\rightarrow$ scan.** Depth pixels are projected through the calibrated camera pose (0.168 m forward, 0.152 m high, 4.09° down). Points with $0.05 \text{ m} \leq \text{height} \leq 0.60 \text{ m}$ are kept and binned at 0.5°.
 - This is `ubot_mono_nav/depth_to_scan`, the same geometry as the camera-only package.
 - A plain centre-row `depthimage_to_laserscan` would report the floor as an obstacle at ~2.1 m, because of the tilt.
- **Mounting.** The LiDAR scan plane is at 0.3627 m, the same for every LiDAR stack. The depth-camera band covers 0.05–0.60 m.

Fairness rules, applied to every stack:

- identical SLAM and Nav2 parameters, except the sensor's own max range;
- identical costmap marking and clearing ranges (2.5 / 3.0 m, below every sensor's range);

- identical controller limits (0.25 m/s, 1.0 rad/s) and the same footprint polygon.

5. Experimental design

Factors

- Stack (4 in sim: REF, MS200, LD06, OAKD; 3 real: MS200, LD06, OAKD) × World (4 sim, 1 real) × Condition (C0-C4) × Seed.
- SLAM update profile: **default** (slam_toolbox scan-matches after 0.5 m or 0.5 rad of motion) and **tight** (0.1 m / 0.1 rad; AMCL also 0.1 m / 0.1 rad). The default is what practitioners run. The tight profile shows how much of each stack's error comes from odometry between updates. It runs as a separate experiment (`sim_core_tight`) with the same seeds, so every trial pairs with its default counterpart. The RPLIDAR A1 was dropped on 2026-10-01 (not used in this study).

Pairing. Within a (world, condition) cell, every stack runs the same seeds 1...N. The seed sets:

- the sensor noise sequence and per-run bias;
- the degradation draws;
- nothing else: missions and walker paths are fixed per world.

So the stacks are compared seed-by-seed on identical missions.

Trial = two phases. These separate mapping error from navigation error.

1. **Mapping.** The robot drives the world's fixed mapping route while slam_toolbox (online async) builds a map.
 - In simulation a ground-truth pure-pursuit follower drives: 0.20 m/s, 0.40 m lookahead, rotate in place when heading error > 60°. Every stack therefore traverses the identical path.
 - Outputs: map, pose graph, SLAM and EKF trajectories.
2. **Navigation.**
 - AMCL localises on *that trial's own map*, with the initial pose at the map origin (the robot starts where mapping started).
 - Nav2 drives the world's fixed goal sequence. Each goal gets a budget of $\max(60 \text{ s}, 4 \times \text{straight-line} / 0.25 \text{ m/s} + 30 \text{ s})$ of sim time.
 - Outputs: per-goal outcome, AMCL trajectory, contacts.

Worlds. Sim worlds are in `ubot_worlds`. Furniture is static, and every world uses the same 2 ms physics step.

ID	World	Size (feasible area)	Why it is in the study	Route	Goals
W0	arena_5x5: sim replica of the physical arena	16.3 m ²	Controlled; sim-to-real anchor	22.2 m	6
W1	small_house (AWS RoboMaker, Harmonic port)	121.9 m ²	Rooms, doorways, clutter	75.6 m	8
W2	bookstore (AWS)	118.5 m ²	Retail aisles; glass storefront above a sill	91.7 m	8
W3	small_warehouse (AWS)	212.8 m ²	Large open space beyond 12 m sensor range; repetitive racking	103.7 m	8

The Harmonic ports come from community pull requests on the archived AWS repositories, as packaged in `turtlebot-maze/tb_worlds` (pinned commit `30424d5`). The hospital world has no Harmonic port and is excluded.

Conditions

I D	Condition	Implementation (sim)	Real-arena equivalent
C 0	nominal	-	-
C 1	glass	Panes with full collision but hidden from the LiDAR and depth camera (<code>visibility_flags</code>), placed across routes. Placements are in <code>missions/<world>.yaml</code> . Validated: a beam that crosses the pane reads the wall behind it (5.21 m vs the pane at 4.87 m)	1.10 × 0.60 m clear acrylic sheet at the same arena position
C 2	dynamic	A 0.4 × 0.4 × 1.7 m walker loops a fixed path at 0.5–0.6 m/s (no collision, visible to sensors)	A person walks the taped line to a metronome (0.5 m/s)
C 3	degraded (dust / dirty window proxy)	<code>scan_model</code> : 20 % dropout, 2 % spurious short returns, noise × 3	Diffusing film over 25 % of the LiDAR window (fixed pattern)
C 4	low light (camera stack only)	Ambient and sun × 0.1, indoor lights off	Room lights off, one lamp

C3 and C4 in the real arena are **exploratory**: the sim and real degradations are not calibrated against each other. They do not enter RQ5.

6. Simulation procedures

6.1 Sensor models

`scan_model` applies, per beam, $\hat{r} = r + b(r) + \varepsilon$, with $\varepsilon \sim N(0, \sigma(r)^2)$ and b drawn once per run from $U(-B(r), B(r))$.

- Returns pushed outside `[range_min, range_max]` become "no return".
- Gazebo produces a noiseless scan at the profile's rate, beam count, field of view and range.
- Interactive sim (`bench:=false`) uses Gazebo's constant- σ noise instead.

6.2 Ground-truth maps (`gt_map`)

- A collision-free, noiseless scanner (2880 beams, 30 m range, three LiDARs at 0.08, 0.22 and 0.3627 m) is teleported through the world. It starts at the spawn point, then visits a 0.5 m lattice breadth-first (0.75 m in the open warehouse, where every surface is seen from many nodes). A lattice node is visited only once it is known free with ≥ 0.30 m clearance, inside the world's bounds, and connected to the spawn through known-free space.
- Cell labels:
 - occupied: ≥ 2 hits and a hit ratio ≥ 0.25 ;
 - free: ≥ 2 passes and not occupied.
- Two outputs:
 - `<world>.yaml`: the 0.3627 m slice, which is the ground truth for map quality;

- `<world>_traversable.yaml`: the union of all three slices, which is what physically stops the robot. It is the ground truth for missions, SPL and clearance.
- No robot, controller or SLAM is involved. The only model is the rendered geometry.

6.3 Missions (`make_missions`, **deterministic**)

All planning happens on the traversability map, with glass panes burned in so that missions are identical in every condition. Feasible cells are free cells with ≥ 0.25 m clearance (the chassis circumscribed radius 0.213 m plus margin), connected to the spawn.

- **Goals.** Geodesic farthest-point sampling from the spawn, among cells with ≥ 0.45 m clearance (0.35 m in the arena). Goals are visited in sampling order, so every leg is long. Headings come from a fixed seed.
- **Mapping route.** Coverage points are placed by geodesic farthest-point sampling until the largest gap is below 1.2 m (arena), 3.0 m (house, bookstore) or 5.0 m (warehouse). They are toured greedily from the spawn and back to it, which gives a loop closure. Consecutive points are joined by clearance-preferring shortest paths over cells with ≥ 0.30 m clearance. The grid path is then straightened locally (each straight segment keeps ≥ 0.35 m clearance and stays within 0.25 m of the grid path) and resampled every 0.10 m.
 - Without straightening, the 8-connected zig-zag made the follower crawl at ~ 0.08 m/s.
 - The extra clearance is needed because the follower cuts corners, and during an in-place turn the rear of the chassis swings ~ 0.33 m about the front-axle pivot (section 4.1). At 0.25 m the pilot's ground-truth follower hit a box.
- A preview PNG is written per world (`missions/<world>.png`).

6.4 Running

```
source ~/uni-bot/install/setup.bash
ros2 run ubot_bench bench run pilot    # then sim_core, sim_core_tight, sim_degradation,
sim_to_real
ros2 run ubot_bench bench list sim_core -v          # progress
~/uni-bot/.venv-bench/bin/python -m ubot_bench.analysis.report sim_core
~/uni-bot/.venv-bench/bin/python -m ubot_bench.analysis.compare sim_core sim_core_tight
```

- Each trial runs headless, in its own `ROS_DOMAIN_ID` and `GZ_PARTITION`, and in its own process group, and is fully killed at the end.
- Infrastructure failures (startup timeout, crash) are retried once. Task failures are data and are never retried.
- A mapping trial whose ground-truth follower makes no progress for 30 s is marked `stuck`. This is a protocol failure, since the follower does not use the sensor. It is reported and excluded, never retried silently.
- `bench run` refuses to start if `ubot_worlds`' installed missions, maps or worlds differ from the source tree (the package installs copies).
- Close any interactive Gazebo before a campaign: it competes for CPU and lowers the real-time factor.
- Results go to `~/uni-bot/bench_results/<experiment>/<trial>/.`

Compute budget (estimated from the pilot; update after it runs):

Campaign	Trial s	Estimated time
sim_core	640	~ 3 min per arena trial and ~ 20 – 30 min per house/bookstore/warehouse trial at real-time factor ≈ 0.3 (pilot); faster on an idle machine

Campaign	Trial s	Estimated time
sim_core_tight	640	Same as sim_core
sim_degradation	~1,120	Arena and house only
sim_to_real	480	Arena only

7. Real-robot procedures

7.1 Arena build (W0)

The sim file `ubot_worlds/worlds/arena_5x5.sdf.xacro` is the arena specification.

- **Floor.** 5.0 × 5.0 m inner area, centred on the origin. A taped 0.5 m grid gives the tape-measure validation points.
- **Walls.** 0.60 m tall (e.g. hardboard). The scan plane at 0.36 m must hit them.
- **Obstacles.** B1 0.40 × 0.40 at (1.00, 1.00); B2 0.60 × 0.30 at (−1.20, 0.60); B3 0.40 × 0.40 at (0.40, −1.30); C1 bin of radius 0.15 at (−0.40, −0.20); P1 partition 1.20 m long at (1.60, −0.30). All are 0.60 m tall.
- **Floor fiducials** at (±2.0, ±2.0) for the overhead-camera extrinsics.
- **Spawn** at (−1.8, −1.8), facing +x, with a physical start jig so the robot starts within ±1 cm and ±1°.

7.2 Ground truth: overhead camera and fiducials

- **Camera.** A wide-angle 1080p camera covering the full 5 × 5 m at ≥ 3.5 m height, or two overlapping cameras. It is fixed to the ceiling and must not be touched between calibration and the end of a session.
- **Markers.** AprilTag 36h11, ≥ 0.15 m, flat on top of the robot at a known offset from `base_footprint`; four floor tags at the surveyed fiducial points.
- **Calibration.**
 - Intrinsic: a checkerboard with ≥ 30 views; record the reprojection RMSE.
 - Extrinsic: a PnP fit to the surveyed floor tags.
- **Validation (per session).** Place the robot on ≥ 20 tape-measured grid points at known headings and report the ground-truth position and heading RMSE. **Acceptance:** position RMSE ≤ 2 cm and heading RMSE ≤ 1°. If it fails, recalibrate before running any trial.
- **Time synchronisation.**
 - The camera and the robot's LAN share a clock (chrony, laptop as server); record the offset, which must be < 5 ms.
 - Measure end-to-end camera latency with an LED flash seen by the camera and logged by the Pi; subtract it from ground-truth stamps.
- **Rate and output.** Ground truth runs at ≥ 20 Hz and is written in TUM format, with the same file layout as `sim (gt.tum)`.

7.3 Per-session and per-run checklists

- **Session.**
 - Ground-truth validation (7.2) passes.

- LiDAR bring-up check: rate, beams per scan and range limits match section 4.2. Record any mismatch.
- IMU still test (2 min): gyro bias $< 0.02^\circ/\text{s}$.
- Wheel calibration: 1 m straight and 360° turn checks within 2 %.
- Pivot point: one 360° in-place turn under the overhead camera. Fit a circle to the ground-truth track and record the rotation centre in the body frame (section 4.1).
- **Run.**
 - Battery within the voltage window (e.g. $\geq 7.6\text{ V}$, 2S), recorded.
 - Robot on the start jig, sensor window clean (except C3), condition set up.
 - Start MCAP recording of all topics.
 - Mapping: drive the route by the scripted teleop replay, which is identical per run; the real robot cannot use a ground-truth follower. Then save the map.
 - Navigation: execute the goals with `mission_runner`.
 - Stop the recording and note anomalies in the run log.
- **Order.**
 - Stacks are run in a counterbalanced order within each block (a Latin square over the stacks), so battery and time-of-day effects do not align with stack.
 - Conditions are blocked.
- **Power.** An INA219 on the battery line (100 Hz) gives energy per mission. The sensor current is measured separately where possible.

8. Measurements

Logged per trial (sim and real, identical names):

- `result.json`: status, durations, per-goal outcomes, contacts.
- Trajectories: `gt.tum`, `odom.tum`, `slam.tum` (mapping) and `amcl.tum` (navigation).
- `map.{yaml,pgm,posegraph,data}`.
- `resources.json`: CPU % and RSS per process group, sampled every 2 s.
- `launch.log`, `runner.log`.
- The experiment's `experiment.yaml`, with the git commit and the models commit.

Metrics (defined in `ubot_bench/analysis/metrics.py` and `collect.py`):

Metric	Definition
ATE RMSE	RMSE of the translation error after Umeyama SE(3) alignment without scale [Sturm 2012]; association $\leq 20\text{ ms}$ (evo 1.31)
ATE (anchored)	RMSE after aligning only the first pose: the drift a robot actually experiences
RPE (1 m)	Mean relative translation (m) and rotation ($^\circ$) error over 1 m path segments
Map precision / recall / F1	Mapped-occupied cells within 10 cm of true occupied / true occupied cells (observable from the route) within 10 cm of mapped-occupied / harmonic mean. The map is placed in the world by the SE(2) alignment of its own trajectory
Wall offset	Mean distance from mapped-occupied cells to the nearest true wall (capped at 1 m)
Free-space IoU	IoU of free cells over the observable region
Success	Ground truth within 0.25 m and 0.30 rad of the goal when Nav2 finishes, and within budget. Never the robot's belief

Metric	Definition
False success	Nav2 reports SUCCEEDED but ground truth fails the success criterion
Position-only success (secondary)	Ground truth within 0.25 m, heading ignored. The final in-place turn to the goal heading shifts the base up to ~0.25 m about the front-axle pivot (section 4.1) for every stack, so this separates "reached the place" from that robot effect
SPL	$(1/N) \sum S_i \cdot \ell_i / \max(p_i, \ell_i)$, where ℓ_i is the shortest feasible path on the traversability map from the leg's actual start and p_i is the driven path length [Anderson 2018]
Time to goal	Sim or wall seconds from dispatch to Nav2 result
Collisions	Chassis collision episodes, counted per leg. Sim: contact sensor on all chassis collisions, where a new episode with an object starts after ≥ 0.5 s without contact (validated by driving into a wall: 1 episode). Real: bumper/IMU spike plus video review
Recoveries	Nav2 <code>number_of_recoveries</code> per leg
Localisation ATE	ATE of the AMCL estimate over the navigation phase
CPU	Mean CPU % (of one core) summed over estimation processes (SLAM, AMCL, EKF, scan chain, Nav2 servers), and RSS
Energy (real)	$\int V \cdot I \, dt$ per mission (J), and mean power (W)

Failure-mode taxonomy (RQ3). Every failed leg is labelled by rule where possible, otherwise by video review with two independent raters and Cohen's κ reported:

Label	Meaning	Rule
F1	localisation divergence	AMCL error > 0.5 m before failure
F2	map defect	goal placed wrong because the map is warped: SLAM ATE > 0.3 m
F3	unseen obstacle	contact with an object absent from the scan (glass, walker)
F4	planner/controller stall	timeout with localisation error < 0.25 m
F5	false success	see metrics
F6	other	-

9. Sample size

9.1 Unit and count

- The unit is one trial (one seed) per (stack, world, condition).
- **Sim:** $N = 20$ seeds per cell.
- **Real:** $N = 20$ runs per (stack, condition) in the arena.

9.2 Rationale

- Success is binary per goal. With 20 runs $\times$ 6–8 goals = 120–160 goals per cell, the Wilson 95% CI half-width is ≤ 8 percentage points at $p = 0.8$.
- For continuous metrics, the paired design removes between-seed variance.

9.3 Power check after the pilot

`analysis/power.csv` estimates, from the pilot, the paired n needed to detect a 20% change of the REF median at $\alpha = .05$ and power .8 (paired t , Wilcoxon ARE 0.955). If any primary metric needs $n > 20$, N is raised for that world before the campaign, and the change is recorded in the change log.

9.4 Pilot log (30 Sep to 1 Oct 2026)

The pilot runs REF and MS200 × arena and house × 3 seeds, on a heavily loaded machine (load ≈ 50 on 12 cores with OBS and an interactive sim running; real-time factor ≈ 0.3). Sim-time results are valid, but several **load- and design-related failures** surfaced and were fixed before any reported campaign:

Issue found	Fix
Mapping route zig-zag (47°/m): follower averaged 0.08 m/s	Local straightening of the grid path (6.3)
Ground-truth follower hit a box (corner cutting plus rear swing in pivot turns)	Route clearance 0.30 / 0.35 m; lookahead 0.30 m; stuck detection (6.4)
Installed missions were stale after regeneration, so trials mixed old and new routes	bench run refuses to start on a stale install (6.4)
Lifecycle activation timed out under load, so AMCL never activated and the runner hung	Bounded startup wait; startup failures are retried once (6.4)
Nav2 behaviour-tree server timeout (20 ms) aborted goals under load, giving fake failures	default_server_timeout 1000 ms, wait_for_service_timeout 5000 ms (nav2_bench.yaml)
Warehouse bounds included a strip outside the walls, reachable through doors	Bounds set just inside the measured outer walls
Bookstore storefront is open at the scan plane (sill below)	Three-slice traversability ground truth (6.2)

Arena results so far (3 seeds; navigation re-run pending after the timeout fix):

- Mapping: SLAM ATE is REF 7.2 cm vs MS200 7.5 cm, RPE per 1 m is 6.2 vs 9.4 cm, and map F1 is 1.00 for both. In a 5 × 5 m arena the online estimate is dominated by motion between SLAM updates and the pivot (section 12), not by sensor noise.
- Power: on the first arena navigation pass, ≈ 11–12 paired seeds detect a 20 % change in success or SPL, so N = 20 is adequate. Recompute this on the clean pilot.
- REF's ground-truth failures at goal G5 were false successes: Nav2 reported success, but the final in-place turn shifted the base about 0.3 m. Hence the position-only secondary metric.

The pilot continues and is then analysed with `python -m ubot_bench.analysis.report pilot`.

Campaigns must run on an otherwise idle machine.

10. Statistical analysis plan (pre-registered)

Primary metrics: SLAM ATE RMSE (RQ1), map F1 (RQ1), SPL (RQ2), success (RQ2), SRCC (RQ5). All others are secondary.

Analysis	Method
Descriptives	Median, IQR and a 10,000-resample percentile-bootstrap 95% CI of the median per cell; success with Wilson 95% CIs
Omnibus	Friedman test across stacks per (world, condition), on complete seeds
Budget vs REF	Wilcoxon signed-rank, paired by seed. Holm correction across all comparisons of one metric. Effect size: Cliff's δ and the median paired difference
Success	Cochran's Q across stacks on per-(seed, goal) outcomes; exact McNemar vs REF (Holm)
Robustness	Linear mixed model: metric ~ stack × condition, random intercept per world (statsmodels MixedLM). Reported, but it does not overrule the non-parametric tests
RQ3	Paired ratio condition / nominal per (world, stack, seed); median and bootstrap CI

Analysis	Method
RQ4	Pareto front on (price, median ATE) and (CPU %, median SPL)
RQ5	SRCC: Pearson r between sim and real medians over (stack $\times$ condition) pairs in the arena, with bootstrap CI; Kendall τ of the stack ranking per metric

- **Significance:** $\alpha = 0.05$ after Holm correction. Effects are interpreted by effect size, not p-values alone.
- **Missing data:** trials lost to infrastructure are retried; persistent failures are reported and excluded pairwise.

11. Data management and reproducibility

- Every experiment records `code_version` (the workspace commit, a dirty flag, and the world-models commit). **Campaigns must run from a clean, committed tree.**
- Sim worlds, missions, ground-truth maps and sensor profiles are version-controlled. The third-party models are fetched at a pinned commit (`ubot_worlds/scripts/fetch_models.sh`).
- **Real data:** MCAP rosbags plus the trial folders, named `<date>_<stack>_<condition>_<run>`, with SHA-256 checksums, backed up to two locations. Release on Zenodo (DOI) with the paper.
- **Pre-registration:** this protocol's sections 1, 5, 8 and 10 go on OSF before the real campaign.

12. Threats to validity

Threat	Mitigation
Reference exists only in simulation	RQ5 measures sim-to-real predictivity for the budget stacks; REF conclusions are conditioned on it
Datasheet noise models are simplified (Gaussian plus a per-run bias; no reflectivity, incidence angle or multipath)	Stated. RQ5 tests whether it matters. The model is one equation, fully reported
Glass modelled as fully invisible	Real glass partially reflects near normal incidence [Weerakoon 2022]; the real acrylic C1 checks the direction of the effect
Gazebo camera ignores exposure and noise in low light, so C4 in sim mostly tests nothing for stereo	Reported as a known simulator limitation; the real C4 is exploratory
Mapping by a ground-truth follower in sim vs teleop replay on the real robot	Identical path within each platform. The path difference between platforms is part of the RQ5 gap
Skid-steer odometry depends on floor friction	The real floor is the same for all stacks; wheel calibration check per session
Wheel-odometry yaw under-reads turns by $\sim 4\%$ in sim	Measured (EKF uses IMU yaw rate, which corrects heading); reported
One SLAM method and one controller	Intentional: the sensor is the variable. A Cartographer ablation is future work
slam_toolbox scan-matches only after 0.5 m or 0.5 rad of motion (its defaults, kept for ecological validity)	Between updates, the online estimate is odometry, which hides sensor differences in small worlds (pilot: REF $\approx$ MS200 in the arena). Decided 2026-10-01: keep the defaults for the main results and run the tight 0.1 m / 0.1 rad profile as a paired ablation (<code>sim_core_tight</code> , section 5)

Threat	Mitigation
Scan-plane height (0.36 m) sees over sills and low shelves (bookstore)	That is real behaviour of this robot; traversability ground truth separates the effect
Physics step (2 ms) chosen for speed	Identical across worlds and stacks; the arena pilot checks that behaviour matches 1 ms
Physics engine	DART (Gazebo default). Bullet-Featherstone was tried and does not turn the skid-steer at all. The engine is a launch parameter (<code>physics:=</code>) so an ablation is possible
Skid-steer pivots about the more-loaded axle in simulation (section 4.1), a sharp Coulomb idealisation	Same for every stack. The real pivot point is measured and reported; the difference is part of the RQ5 gap

13. Paper plan (IROS 2027, 6 + 2 pages)

Working title: *What Do You Lose With a \$60 LiDAR? A Paired Sim-and-Real Benchmark of Low-Cost Sensing for Indoor Navigation.*

§	Content	Pages
I	Introduction: access gap; question; contributions	0.75
II	Related work (section 3)	0.5
III	Benchmark: robot, stacks (table), worlds, conditions, ground truth, missions (figure: worlds with routes)	1.25
IV	Protocol and metrics (compact table; pointer to this document)	0.5
V	Results: RQ1 (ATE and map-F1 figure), RQ2 (success and SPL figure, false success), RQ3 (degradation heatmap, taxonomy table), RQ4 (cost Pareto figure), RQ5 (sim-vs-real scatter with SRCC)	2.25
VI	Discussion: guidance for practitioners, limitations	0.5
VI	Conclusion and artifact release	0.25

- **Figures** (from `analysis/figures.py`): `slam_ate`, `map_f1`, `success_rate`, `spl`, `cost_vs_accuracy`, `degradation_ate`, plus the sim-vs-real scatter and the worlds panel (added once there is real data).
- **Video (3 min):** the arena run with a live map; side-by-side trajectory error of two stacks; a visitor moves an obstacle to trigger replanning. The backup is a recorded run plus a Gazebo replay of the same mission.

14. Timeline

Date	Milestone	Exit criterion
30 Sep – 12 Oct 2026	Sim benchmark built; pilot run	Pilot results valid (section 9.3); code tagged <code>bench-v1.0</code>
13 Oct – 31 Oct	<code>sim_core</code> and <code>sim_core_tight</code> campaigns; arena build; ground-truth rig	1,280 trials done; ground-truth validation $RMSE \leq 2$ cm
1 Nov – 20 Nov	<code>sim_degradation</code> ; real bring-up per stack; OSF pre-registration	Bring-up checklist passed per stack

Date	Milestone	Exit criterion
21 Nov – 20 Dec	Real campaign, block 1 (C0, C1)	20 runs per stack and condition
4 Jan – 24 Jan 2027	Real campaign, block 2 (C2; exploratory C3, C4); sim_to_real	-
25 Jan – 14 Feb	Analysis, figures, taxonomy labelling	All RQ figures final
15 Feb – 1 Mar	Writing, video, Zenodo release	Submitted by 1 Mar 2027 (IROS)

Risks and fallbacks:

- **Ground truth inaccurate:** validate every session and fall back to tape-measured waypoints.
- **LiDAR driver problems:** use a supported model and keep a spare.
- **Wheel slip:** log it and report it as a finding.
- **Real campaign slips:** submit with sim results plus a reduced real set (C0 only). RQ5 remains answerable with 2 stacks × 1 condition, weakly.

References

- [Anderson 2018] P. Anderson et al., "On Evaluation of Embodied Navigation Agents," arXiv:1807.06757, 2018.
- [Du 2025] Y. Du, S. Feng, C. G. Cort, P. A. Vela, "Task-driven SLAM Benchmarking For Robot Navigation," IROS 2025 (arXiv:2409.16573).
- [Grupp 2017] M. Grupp, "evo: Python package for the evaluation of odometry and SLAM," <https://github.com/MichaelGrupp/evo>, 2017.
- [Kadian 2020] A. Kadian et al., "Sim2Real Predictivity: Does Evaluation in Simulation Predict Real-World Performance?" IEEE RA-L, 2020 (arXiv:1912.06321).
- [Kümmerle 2009] R. Kümmerle et al., "On measuring the accuracy of SLAM algorithms," Autonomous Robots 27:387–407, 2009, doi:10.1007/s10514-009-9155-6.
- [Macenski 2020] S. Macenski, F. Martín, R. White, J. Clavero, "The Marathon 2: A Navigation System," IROS 2020 (arXiv:2003.00368).
- [Macenski 2021] S. Macenski, I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," JOSS 6(61):2783, 2021, doi:10.21105/joss.02783.
- [Perille 2020] D. Perille, A. Truong, X. Xiao, P. Stone, "Benchmarking Metric Ground Navigation," IEEE SSRR 2020 (arXiv:2008.13315).
- [Sturm 2012] J. Sturm, N. Engelhard, F. Endres, W. Burgard, D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," IROS 2012, doi:10.1109/IROS.2012.6385773.
- [Weerakoon 2022] L. Weerakoon, G. S. Herr, J. Blunt, M. Yu, N. Chopra, "Cartographer_glass: 2D Graph SLAM Framework using LiDAR for Glass Environments," arXiv:2212.08633, 2022.
- [Weerakoon 2024] K. Weerakoon, A. J. Sathyamoorthy, M. Elnoor, A. Zore, D. Manocha, "TOPGN: Real-time Transparent Obstacle Detection using Lidar Point Cloud Intensity for Autonomous Robot Navigation," arXiv:2408.05608, 2024.
- [Ziębiński 2025] A. Ziębiński, P. Biernacki, "How Accurate Can 2D LiDAR Be? A Comparison of the Characteristics of Calibrated 2D LiDAR Systems," Sensors 25(4):1211, 2025, doi:10.3390/s25041211.
- Sensor datasheets: Hokuyo UST-10LX specification; Oradar MS200 user manual PD-P2117008 A0 (2023); LDROBOT LD06 datasheet; Luxonis OAK-D Lite documentation. Links are in Sources.

Sources

Every external source this study relies on, with its link. The papers were verified by web search on 30 September 2026; the links were current on that date.

Papers

Ref	Link
Anderson 2018 (SPL)	https://arxiv.org/abs/1807.06757
Du 2025 (TaskSLAM-Bench)	https://arxiv.org/abs/2409.16573
Kadian 2020 (Sim2Real predictivity, SRCC)	https://arxiv.org/abs/1912.06321
Kümmerle 2009 (SLAM accuracy)	https://link.springer.com/article/10.1007/s10514-009-9155-6
Macenski 2020 (Nav2, The Marathon 2)	https://arxiv.org/abs/2003.00368
Macenski 2021 (SLAM Toolbox)	https://joss.theoj.org/papers/10.21105/joss.02783
Perille 2020 (BARN)	https://arxiv.org/abs/2008.13315
Sturm 2012 (TUM RGB-D, ATE/RPE)	https://cvg.cit.tum.de/_media/spezial/bib/sturm12iros.pdf
Weerakoon 2022 (Cartographer_glass)	https://arxiv.org/abs/2212.08633
Weerakoon 2024 (TOPGN)	https://arxiv.org/abs/2408.05608
Ziębiński 2025 (calibrated 2D LiDAR accuracy)	https://doi.org/10.3390/s25041211

Sensor datasheets (the values in section 4.2 and `sensor_profiles.yaml`)

Sensor	Link
Hokuyo UST-10LX specification	https://www.hokuyo-aut.jp/dl/UST-10LX_Specification.pdf
Oradar MS200 user manual (Table 2-1)	https://www.orbbec.com/wp-content/uploads/2023/07/MS200-dToF-Lidar-user-manual-A0-20230605-1.pdf
LDROBOT LD06 datasheet	https://www.inno-maker.com/wp-content/uploads/2020/11/LDROBOT_LD06_Datasheet.pdf
Luxonis OAK-D Lite	https://docs.luxonis.com/hardware/products/OAK-D%20Lite
BNO085 gyro noise (E2 still test, internal)	<code>ubot_bringup/config/bno085_params.yaml</code> , commit 6ab133d

Simulation assets

Item	Link
World list that motivated the choice (Gazebo Classic)	https://automaticaddison.com/useful-world-files-for-gazebo-and-ros-2-simulations/
Harmonic ports used (pinned commit 30424d5)	https://github.com/zp78-ship-it/turtlebot-maze
AWS RoboMaker small house (Harmonic port: PR #47)	https://github.com/aws-robotics/aws-robomaker-small-house-world/pull/47
AWS RoboMaker small warehouse (Harmonic port: PR #27)	https://github.com/aws-robotics/aws-robomaker-small-warehouse-world/pulls
AWS RoboMaker bookstore	https://github.com/aws-robotics/aws-robomaker-bookstore-world
AWS RoboMaker hospital (no Harmonic port; excluded)	https://github.com/aws-robotics/aws-robomaker-hospital-world

Software

Tool	Link
Nav2	https://github.com/ros-navigation/navigation2
slam_toolbox	https://github.com/SteveMacenski/slam_toolbox
robot_localization	https://github.com/cra-ros-pkg/robot_localization
gz_ros2_control	https://control.ros.org/jazzy/doc/gz_ros2_control/doc/index.html
evo 1.31.1	https://github.com/MichaelGrupp/evo
statsmodels	https://www.statsmodels.org

Venue dates

Venue	Link
ICRA 2027 call for papers (deadline 16 Sep 2026, passed)	https://2027.ieee-icra.org/announcements/call-for-technical-papers/
IROS 2027 deadline (1 Mar 2027)	https://mldeadlines.com/conference/iros-2027/

Change log

Date	Change	Reason
2026-09-30	v1.0	Initial protocol
2026-09-30	v1.1	Added the measured pivot behaviour (4.1), bumper and glass validation, physics-engine note, pivot measurement on the real robot (7.3). Sim now drives all four wheels explicitly (as the ESP32 does); wheel friction direction set to the axle
2026-10-01	v1.2	Pilot fixes (9.4): route straightening and clearance, stuck detection, stale-install guard, bounded Nav2 startup, Nav2 server timeouts, warehouse bounds and spacing, position-only success metric
2026-10-01	v1.3	RPLIDAR A1 dropped (not used). Tight SLAM profile (0.1 m / 0.1 rad) added as a paired ablation (<code>sim_core_tight</code>). Sources section with links added